VK7SPORTS

BACKEND IMPROVEMENT & UPGRADE MANUAL

Database / ERD Review • Production Readiness • Security • Commerce • B2B • Operations

Prepared for	Backend Intern / Development Team
Review basis	VK7SPORTS ER Diagram + 25-page Database Schema Specification
Review date	10 August 2026
Document purpose	Actionable instruction manual for improvements, upgrades, corrections and implementation sequencing

Important scope note

The submitted material is a database/ERD design, not the complete backend source code. This manual therefore audits the data model and defines the backend work that should exist around it. Items such as controllers, API routes, authentication implementation, tests, deployment, queues and payment webhooks must be verified in the actual repository before being marked complete.

1. Executive Assessment

The intern has produced a useful first-pass commerce schema covering identity/RBAC, catalog, variants, carts, orders, payments, custom kits, institutional enquiries, reviews, support, CMS banners and activity logs. The structure is understandable and already includes several good practices such as UUIDs for transactional entities, unique emails/SKUs/slugs, CHECK constraints on quantities and ratings, order address snapshots, audit logging, and product-level tax/HSN fields.

However, the design should not yet be treated as production-ready for a serious e-commerce + institutional sportswear platform. The largest gaps are inventory architecture, coupon redemption control, payment/refund lifecycle, order status history, shipping/fulfilment, B2B quotations, custom-kit normalization, data integrity constraints, timestamps/auditing, soft deletion, security/session design, and operational scalability.

Area	Current assessment	Priority
Core relational structure	Good foundation; relationships are understandable.	Keep + refine
Production data integrity	Several nullable dual-reference fields and missing cross-table rules.	P0
Inventory	Single stock_quantity on variant conflicts with stated multi-warehouse scope.	P0
Payments/refunds	Basic payment row only; insufficient for gateway webhooks/refunds/reconciliation.	P0
Orders/fulfilment	No status history, shipment entity, cancellation/return workflow.	P0
Security/auth	RBAC table exists, but sessions/tokens/login security are not represented.	P0
B2B/institutional	Enquiry exists, but quotation/version/approval/order conversion is thin.	P1
Custom kit builder	Useful concept, but player roster in JSONB will become hard to validate/query.	P1
Observability/audit	activity_logs exists but needs richer immutable event data.	P1
Performance/indexing	Some indexes mentioned; full index/constraint strategy is missing.	P1
Backend API/tests/deployment	Not evidenced by submitted files.	Repository verification required

2. What Is Already Done Well

- Clear domain separation: identity, catalog, shopping, transactions, custom teamwear/B2B, customer service, CMS and audit.
- Order addresses are stored as JSON snapshots, preventing later profile edits from rewriting historical orders.
- Variant SKUs are separated from base product SKUs.
- The schema uses CHECK constraints for stock, quantity, rating and positive prices in several important places.
- Unique identifiers are planned for order numbers, ticket numbers, invoice numbers and payment transaction references.
- Custom kit designs can be attached to carts and orders, which is directionally correct for mixed standard/custom purchases.
- Activity logs are included from the beginning rather than added after launch.

- Product tax rate and HSN code fields acknowledge Indian tax requirements.

3. Critical P0 Changes - Must Be Done Before Production

3.1 Replace Single-Column Inventory with Multi-Warehouse Inventory

The specification says the platform supports multi-warehouse inventory, but product_variants contains only stock_quantity. That is a single global balance and cannot safely represent multiple warehouses, reservations, damaged stock, incoming stock or allocation during checkout.

- Create warehouses(id, code, name, address fields, is_active, created_at, updated_at).
- Create inventory_levels(id, warehouse_id, product_variant_id, on_hand, reserved, available, reorder_level, updated_at) with UNIQUE(warehouse_id, product_variant_id).
- Create inventory_movements(id, warehouse_id, product_variant_id, movement_type, quantity_delta, reference_type, reference_id, reason, actor_user_id, created_at).
- Do not decrement inventory only from application memory. Use a database transaction/locking strategy during checkout and reservation.
- Define reservation expiry so abandoned carts do not permanently block stock.
- Remove or demote product_variants.stock_quantity after migration; if retained as a cache, make the source-of-truth rule explicit.

3.2 Enforce Cart/Order Line Item Integrity

cart_items and order_items allow both product_variant_id and custom_design_id to be NULL, or both to be populated. That creates invalid or ambiguous line items.

- Add a CHECK constraint requiring exactly one merchandise reference: (product_variant_id IS NOT NULL) XOR (custom_design_id IS NOT NULL).
- Add UNIQUE(cart_id, product_variant_id) for normal variants where appropriate, or define merge behavior in service logic.
- Store product title/SKU/customization snapshot on order_items so historical invoices remain correct if catalog data changes.
- Add tax_rate_snapshot, hsn_code_snapshot, discount_amount, tax_amount and final_line_total to order_items if tax/invoice reporting is line based.
- Never calculate authoritative totals only on the frontend.

3.3 Build a Real Payment, Refund and Webhook Model

The payments table captures a transaction reference and status, but modern gateway flows are asynchronous and may contain attempts, captures, failures, partial refunds and webhook retries.

- Separate payment attempts from refunds. Add payment_gateway, gateway_order_id/payment_intent_id, currency, gateway_status, failure_code, failure_message, captured_at, created_at, updated_at.
- Create payment_webhook_events with gateway_event_id UNIQUE, payload JSONB, received_at, processed_at, processing_status. This provides idempotency.
- Create refunds(id, payment_id, order_id, amount, reason, gateway_ref, status, initiated_by, created_at, completed_at).
- Do not store a refund amount only inside support_tickets. A support ticket may request a refund; the refund ledger must be financial and auditable.

- Implement idempotency keys for checkout/payment creation so retries cannot create duplicate orders or charges.
- Verify signatures for Razorpay/Stripe/etc. server-side; never trust a frontend 'payment successful' response.

3.4 Add Order Lifecycle History and Fulfilment

orders has one current status and tracking fields. This loses the history of who changed an order and does not support split shipments or multiple tracking numbers.

- Create order_status_history(order_id, from_status, to_status, actor_user_id, reason, created_at).
- Create shipments(id, order_id, warehouse_id, courier_name, tracking_number, status, shipped_at, delivered_at, created_at).
- Create shipment_items(shipment_id, order_item_id, quantity).
- Define a strict order state machine and reject illegal transitions.
- Add cancelled_at, cancellation_reason and optional cancelled_by.
- Make fulfilment updates transactional with inventory movement records.

3.5 Authentication and Account Security

roles/users are present, but no session/token/authentication lifecycle is described in the submitted schema.

- Use Argon2id or bcrypt with a strong work factor for password hashes; never store plaintext passwords.
- Add email/phone verification tokens with expiry and one-time use.
- Add password_reset_tokens with hashed token, expires_at, used_at.
- Use secure server-side sessions or short-lived access tokens plus rotating refresh tokens; store refresh token hashes, device/session metadata and revocation state.
- Add failed_login_count / lockout or rate-limit controls at the application/infrastructure layer.
- Require MFA for super_admin and privileged back-office roles.
- Do not put authorization only in frontend route guards. Every privileged API must enforce permissions server-side.
- Audit login, failed login, password reset, role change, refund approval and inventory adjustment.

3.6 Database Constraints, Cascades and Nullability

Foreign keys are shown, but delete/update behavior is not defined. Production behavior must be explicit.

- Define ON DELETE behavior for every FK. Transactional history (orders/payments/audit) should normally be RESTRICT/NO ACTION, not cascade-delete.
- Use CASCADE for clearly dependent ephemeral rows only where safe (for example cart_items when a cart is deleted).
- Add CHECK(valid_until > valid_from) for coupons.
- Add CHECK(sale_price IS NULL OR sale_price > 0) and business rule for sale_price versus base_price.
- Add CHECK(discount_value > 0); percentage coupons should also be constrained to <= 100.
- Add non-negative checks for all monetary totals, shipping costs, refund amounts and additional_price.
- Use timestamptz (PostgreSQL) rather than timezone-naive timestamp for business events.
- Add updated_at consistently to mutable tables.

4. P1 Architecture Upgrades

Coupons & Promotions

- Add is_active and per-user usage limits.
- Create coupon_redemptions(coupon_id, user_id, order_id, discount_amount, redeemed_at) with uniqueness/idempotency rules.
- Support applicability rules: product/category/sport/user segment/min quantity.
- Do not rely only on a global usage_limit counter without a redemption ledger.

B2B Quotations & Institutional Ordering

- Split bulk enquiry from quotation. Create quotations and quotation_items.
- Version quotations so revised prices do not overwrite earlier offers.
- Add quotation status, validity date, tax, shipping, terms, created_by, approved_by and conversion_to_order reference.
- Consider organizations + organization_members instead of storing organization_name/tax_number directly on every user when multiple contacts belong to one institution.
- Support purchase order number, GST billing data and approval workflow.

Custom Kit Builder

- Move player_roster from JSONB into custom_kit_players(design_id, player_name, jersey_number, size, personalization fields) if roster operations/validation are important.
- Create custom_design_assets for logos/artwork and retain file metadata, MIME type, storage key, moderation/scan status.
- Add design_version/status (draft, submitted, proof_ready, approved, locked) and approval timestamps.
- Store pricing components for customization rather than deriving old orders from the latest pricing rules.

Returns, Exchanges & Support

- Create returns/return_items instead of embedding return_reason and refund_amount into generic support_tickets.
- Represent exchange, refund, replacement and rejection outcomes explicitly.
- Link returns to order/order_item and shipment.
- Add ticket messages/comments so support is a conversation, not only a ticket header.

Catalog & Merchandising

- Add product.updated_at and optional deleted_at/archived_at.
- Consider many-to-many sports/categories if products can belong to multiple taxonomies.
- Create normalized product_attributes/variant_options if size/color will expand to gender, fit, sleeve, material, etc.
- Add media storage metadata and validation; do not accept arbitrary URLs without controls.
- Add SEO metadata only if required by frontend/CMS: meta_title, meta_description, canonical fields.

Audit & Admin

- Expand activity_logs with entity_type, entity_id, action, before_data/after_data or metadata JSONB, request_id, user_agent and created_at.
- Treat audit logs as append-only; normal admins should not be able to edit/delete them.
- Add admin notes/approval records for sensitive actions.
- Never log passwords, access tokens, payment card data or secrets.

5. Recommended Revised Data Model

Domain	Keep	Add / Change
Identity	roles, users, user_addresses	organizations, organization_members, sessions/refresh_tokens, verification/reset tokens, optional MFA records
Catalog	sports, categories, products, product_variants, product_media	timestamps, soft archive, richer variant options if needed
Inventory	product_variants	warehouses, inventory_levels, inventory_movements, stock_reservations
Cart	carts, cart_items	timestamps, currency, cart status/expiry, XOR line-item constraint
Promotions	coupons	coupon_redemptions, applicability rules
Orders	orders, order_items	status history, immutable snapshots, cancellation fields
Fulfilment	tracking fields on order	shipments, shipment_items
Payments	payments	webhook events, payment attempts/metadata, refunds
Custom kits	custom_kit_designs	players, assets, versions/approval, pricing snapshot
B2B	bulk_enquiries	quotations, quotation_items, versions, conversion to order
Service	product_reviews, support_tickets	ticket messages, returns, return_items
CMS	homepage_banners	optional campaign scheduling and audit fields
Audit	activity_logs	richer append-only metadata/request tracing

6. API & Backend Service Requirements

The submitted files do not prove that these APIs/services exist. The intern should use this as an implementation checklist and mark each item only after repository verification.

Service area	Minimum backend capability
Auth	register, login, logout, refresh/revoke session, verify email/phone, forgot/reset password, profile/address management
Catalog	sports/categories, product listing/detail, filters/search, media, availability
Cart	create/get cart, add/update/remove item, validate stock, apply/remove coupon, totals preview
Checkout	address validation, shipping method, stock reservation, authoritative totals, create order
Payments	create gateway order/intent, webhook handler, payment status, refund initiation/status
Orders	customer history/detail, cancel request, admin state transitions, shipment tracking
Custom kit	draft/save design, upload asset, roster CRUD, preview/proof status, approval
B2B	submit enquiry, admin review, quotation CRUD/version, approve/reject, convert to order
Reviews	submit, list approved, admin moderation
Support/returns	ticket CRUD/messages, return request, item inspection outcome, refund/exchange
Admin	catalog, inventory adjustments, orders, refunds, coupons, banners, roles/permissions, audit

7. Security Rules the Intern Must Follow

- Validate every request server-side using a schema validator; reject unknown/invalid types and excessive lengths.
- Use parameterized ORM/query-builder operations; no string-concatenated SQL.
- Use least-privilege database credentials and separate production secrets from source control.
- Encrypt transport with HTTPS/TLS; secure cookies must use HttpOnly, Secure and appropriate SameSite settings.
- Rate-limit login, OTP, password reset, coupon checks, review submission and public enquiry endpoints.
- Use CSRF protection when authentication is cookie-based.
- Apply strict CORS allowlists in production.
- Validate uploads by MIME/signature/size, generate safe storage keys, scan risky uploads and keep object storage private unless public access is intentional.
- Never store card PAN/CVV. Use the payment gateway's hosted/tokenized flow.
- Protect admin APIs with explicit permissions and MFA for sensitive operations.
- Return generic authentication errors; do not reveal whether a specific email exists where that enables enumeration.
- Centralize error handling and never leak stack traces, SQL errors, secret keys or internal paths to clients.

8. Performance & PostgreSQL Checklist

- Index every frequently queried foreign key: users.role_id, addresses.user_id, products.sport_id/category_id, variants.product_id, media.product_id, cart_items.cart_id, order_items.order_id, payments.order_id, reviews.product_id/user_id, etc.
- Add composite indexes based on real queries, e.g. products(is_active, sport_id, category_id), orders(user_id, created_at DESC), inventory_levels(product_variant_id, warehouse_id).
- Use partial indexes for active/published records where beneficial.
- Use case-insensitive email handling consistently (e.g. normalized lowercase or PostgreSQL citext).
- Use pagination for products, orders, logs, tickets and reviews; avoid unbounded list endpoints.
- Use EXPLAIN ANALYZE on slow production-like queries before adding speculative indexes.
- Use connection pooling and define transaction boundaries for checkout, stock allocation and refunds.
- Do not use JSONB for data that requires frequent relational filtering, uniqueness or referential integrity.

9. Testing & Quality Gate

Test layer	Required coverage
Unit	pricing, discount rules, tax calculations, state transitions, permission checks, inventory math
Integration	database constraints, repositories/services, transaction rollback, concurrent stock reservation
API	auth failures, validation, RBAC, happy paths, edge cases, pagination
Payment	valid/invalid webhook signatures, duplicate webhook idempotency, failed payment, partial/full refund
Checkout concurrency	two customers attempting last unit; exactly one succeeds
Security	IDOR/BOLA checks, privilege escalation, rate limits, upload validation, secret leakage
Migration	upgrade from previous schema, rollback strategy, seed data, constraint validation
E2E	register -> browse -> cart -> checkout -> payment -> shipment;

	B2B enquiry -> quotation; custom kit flow

Minimum release gate: all migrations reproducible from an empty database, automated tests green, lint/type checks green, no high/critical dependency vulnerabilities, staging smoke test complete, backup/restore tested, and API documentation current.

10. Migration Plan - Do Not Rewrite Production Data Blindly

1. Create a new branch and migration set; never manually edit a production database as the primary method.
2. Back up database and test restoration before structural changes.
3. Add new tables/columns first, backfill data, then introduce NOT NULL/UNIQUE/CHECK constraints after validation.
4. For inventory, migrate variant stock into a default warehouse inventory_levels row, validate totals, then switch services to the new source of truth.
5. For refunds/returns, preserve existing support ticket information and link migrated financial records where possible.
6. Deploy backwards-compatible application code before removing old columns.
7. Only drop/deprecate old stock/tracking/refund fields after production verification and a rollback window.

11. Intern Execution Roadmap

Phase	Deliverable	Acceptance criteria
Phase 0 - Repository audit	Map existing backend code to this schema/manual.	Architecture diagram, routes/services list, migrations list, test status; no guessing.
Phase 1 - Data integrity	P0 constraints, timestamps, FK delete rules, order line snapshots.	Migration passes on clean DB and representative seed data.
Phase 2 - Inventory	Warehouse/levels/movements/ reservations.	Concurrent last-unit test passes; audit trail exists.
Phase 3 - Payments	Gateway intent/order, webhooks, idempotency, refunds.	Duplicate webhooks do not duplicate financial effects.
Phase 4 - Orders/fulfilment	Status history, shipment entities, cancellation.	Illegal transitions blocked; split shipment supported if required.
Phase 5 - Security	Sessions/tokens, reset/verification, admin MFA/RBAC, rate limits.	Security test checklist passes.
Phase 6 - B2B/custom kits	Quotation model, roster/assets/version workflow.	Quotation revision and design approval are auditable.
Phase 7 - Support/returns	Ticket messages + returns/refund linkage.	Return is linked to order items and financial refund.
Phase 8 - Performance/ops	Indexes, logs, monitoring, backup/restore, API docs.	Staging load/smoke test + restore drill complete.

12. Definition of Done for the Intern

- Every schema change is represented by versioned migrations.
- Every table/field has a documented purpose and constraints.
- Every privileged operation has server-side authorization.
- Every financial/inventory mutation is transactional and auditable.
- Payment/webhook operations are idempotent.
- API validation and consistent error responses are implemented.

- Critical flows have automated tests, including concurrency and failure paths.
- OpenAPI/Postman documentation matches the implemented API.
- Environment variables are documented in .env.example without secrets.
- Production deployment has health checks, structured logs, monitoring and database backup/restore instructions.
- No task is marked complete solely because a table exists; the service logic, validation, authorization and tests must also be complete.

13. Specific Corrections to the Submitted Schema

Submitted element	Problem	Required change
product_variants.stock_quantity	Does not satisfy the document's claimed multi-warehouse capability.	Replace with warehouse inventory model.
cart_items product_variant_id/custom_design_id	Both nullable; invalid zero-reference or dual-reference rows possible.	Add exactly-one CHECK constraint.
order_items product_variant_id/custom_design_id	Same integrity issue and insufficient historical snapshot.	Add XOR constraint + SKU/title/tax/HSN/customization snapshots.
coupons	No redemption ledger or per-user usage enforcement.	Add coupon_redemptions and rules.
payments	Refund represented only by payment_status and ticket refund_amount.	Add webhook event and refunds ledger.
orders tracking_number/courier_name	Only one shipment can be represented.	Add shipments + shipment_items.
orders.order_status	Current state only; no history.	Add order_status_history.
bulk_enquiries quoted_price/quote_pdf_url	Enquiry and quotation concerns are mixed.	Create quotation entities/versioning.
custom_kit_designs.player_roster JSONB	Convenient initially but weak for roster validation/reporting.	Normalize if roster is operationally important.
support_tickets return/refund fields	Support conversation and return workflow are conflated.	Create ticket_messages + returns/return_items.
activity_logs	Too little context for forensic/audit use.	Add entity/request/metadata fields; append-only.
timestamps	Mutable tables are inconsistent; timezone not specified.	Use created_at/updated_at and timestamptz consistently.

14. Final Instruction to the Backend Intern

Do not start by adding random tables. First audit the current repository and map every existing model, migration, endpoint, service and test to this manual. Then implement the P0 items in migration-safe phases. Preserve working behavior, add tests before destructive changes, and provide a short evidence report for every phase: migration names, API endpoints changed, tests added, screenshots/logs from staging, and known limitations.

The goal is not merely a larger schema. The goal is a backend that protects money, stock, customer data and operational history under retries, concurrency, failures and admin actions.

Appendix A - Evidence Used for This Review

1. VK7SPORTS ER Diagram: two-page dbdiagram.io relationship diagram showing the submitted tables and cardinalities.

2. VK7SPORTS - E-Commerce Database Schema Specification: 25-page table-by-table specification covering RBAC, users, addresses, sports/categories/products/variants/media, carts, coupons, orders/items/payments, custom kits, bulk enquiries, wishlists/reviews/support, banners and activity logs.

No backend source repository, API implementation, migrations, deployment configuration or automated test suite was included in the submitted files; those areas are therefore specified as required verification/implementation work rather than asserted defects in code.